

Best Practices für die Notebook-Entwicklung

Diese Seite fasst Softwareentwicklungs-Empfehlungen für professionelle, wartbare Notebook-Projekte zusammen: von Versionskontrolle über Testautomatisierung bis zu CI/CD. Sie richtet sich vor allem an Teams, die über einfache Einzelanalysen hinaus produktionsreife Workflows aufbauen wollen.

Versionskontrolle mit Git

Der Workspace sollte über **Databricks Git-Ordner** mit einem Remote-Repository (GitHub, GitLab usw.) verbunden werden. Die Anmeldung erfolgt über Personal Access Tokens mit passenden Rechten (bei GitHub z. B. die Scopes "repo" und "workflow"). Wichtig: Entwickler sollten auf eigenen **Feature-Banches** arbeiten, getrennt vom Hauptbranch (main) – das ist eine grundlegende Softwareentwicklungs-Praxis, die sichere Weiterentwicklung ermöglicht, bevor Änderungen in die Produktion übernommen werden.

Code modularisieren

Statt die gesamte Logik in Notebooks zu bündeln, sollten wiederverwendbare Funktionen in eigene Module ausgelagert werden – zum Beispiel eine Datei `transforms.py` in einem Ordner `covid_analysis`. Das ermöglicht das Teilen von Code zwischen mehreren Notebooks, ohne ihn zu duplizieren, und verbessert Wartbarkeit und Konsistenz. Typische ausgelagerte Funktionen: Filtern, Pivotieren, Datenbereinigung, Spaltenmanipulation.

Abhängigkeiten verwalten

Paketabhängigkeiten sollten in einer `requirements.txt`-Datei im Repository-Wurzelverzeichnis dokumentiert werden, mit exakt fixierten Versionen (z. B. `pandas==1.4.2`, `pytest==7.1.2`) statt automatischer Updates. Das sorgt für reproduzierbares Verhalten über Zeit und verschiedene Umgebungen hinweg und verhindert unerwartete Fehler durch Bibliotheks-Updates.

Testen

Das Framework **pytest** wird empfohlen, um gemeinsam genutzten Code unabhängig vom Notebook zu testen. Getrenntes Testen auf Modulebene deckt Fehler auf, bevor sie ein umfangreiches Hauptnotebook zum Scheitern bringen. Konkrete Beispiele aus der Dokumentation:

- Testdaten als CSV-Fixture bereitstellen
- Assertions zur Prüfung von Filter- und Transformationslogik
- Tests für Spaltennamen-Bereinigung und Datentyp-Umwandlung
- Ein eigenes Test-Notebook, das importierte Tests ausführt und Erfolgs-/Fehlerkennzahlen ausgibt

Jobs statt manuelles Ausführen

Statt Notebooks manuell zu starten, sollten **Databricks Jobs** mit definierten Aufgaben-Abhängigkeiten eingerichtet werden. Jobs sollten Notebooks aus dem committeten Branch (in der Regel "main") laden, nicht aus dem lokalen Arbeitsstand im Workspace. Ein Beispiel-Job führt zunächst Tests aus, dann erst das eigentliche Analyse-Notebook – Verifikation vor Produktionsausführung. Jobs lassen sich manuell auslösen oder nach Zeitplan automatisieren.

CI/CD mit GitHub Actions

Für Continuous Integration/Continuous Deployment wird ein GitHub-Actions-Workflow empfohlen, der bei Pull Requests automatisch ausgelöst wird. Statt persönlicher Workspace-Zugangsdaten sollte GitHub ein Databricks-Zugriffstoken erhalten, das mit einem **Service Principal** verknüpft ist (ein technisches, nicht-personengebundenes Konto). So bleibt Zugriffskontrolle gewahrt, während Tests automatisch bei jeder Codeänderung laufen.

Empfohlene Repository-Struktur

```
├─ covid_analysis/  
│   └─ transforms.py  
├─ notebooks/  
│   ├── covid_eda_modular  
│   ├── covid_eda_raw  
│   └─ run_unit_tests  
├─ tests/  
│   └─ testdata.csv
```

```
| └─ transforms_test.py
└─ requirements.txt
```

Diese Struktur trennt Verantwortlichkeiten sauber: Analyse-Notebooks, wiederverwendbare Module und Tests liegen jeweils in eigenen Ordnern. Das erleichtert Navigation, Wartung und das Verständnis der Projektabsicht.

Code-Review und Zusammenarbeit

Pull Requests sind als verpflichtende Kontrollpunkte vorgesehen, bevor Änderungen in den Hauptbranch übernommen werden. Der übliche Ablauf: Branch erstellen, Änderungen mit aussagekräftigen Commit-Nachrichten committen, zum Remote-Repository pushen, Pull Request zur Peer-Review eröffnen.

Sicherheit

Persönliche Workspace-Zugangstoken sollten niemals mit externen Diensten geteilt werden. Stattdessen sollten Organisationen dedizierte Service Principals für CI/CD-Integrationen anlegen – nach dem Prinzip der minimalen Rechtevergabe (Least Privilege).

Performance und Reproduzierbarkeit

Durch die Nutzung committeter Versionen aus dem Git-Repository statt lokaler Workspace-Kopien liefern Jobs reproduzierbare Ergebnisse auf Basis eines definierten Code-Standes. Zusätzlich sorgt die Angabe exakter Cluster-IDs und -Versionen in der Job-Konfiguration für konsistente Rechenumgebungen über mehrere Ausführungen hinweg.

Zusammengefasst: Git-Workflow, Code-Modularisierung, fixierte Abhängigkeiten, automatisierte Tests, Jobs statt manueller Ausführung sowie CI/CD-Absicherung mit Service Principals bilden gemeinsam die Grundlage für professionelle, produktionsreife Notebook-Projekte.

Quelle: <https://docs.databricks.com/aws/en/notebooks/best-practices>
Stand: 2026-08-06